2026 Winter Classic Invitational Student Cluster Competition

ORNL "Mystery" Challenge: Quantum Transfer Learning

Hands-on with Odo Report

Not So Slow Slugs Team 1

April 4, 2026

Contents

Task 1. Parameter Sweep

1.1. GPU Count Benchmark (1–8 GPUs, `lightning.kokkos`, 30 epochs)

We benchmarked the QML transfer learning code on a single compute node using 1 through 8 MPI tasks (GPUs). Results are summarized in Table 1.

Table 1: Training results across GPU counts (30 epochs, `lightning.kokkos` device).

GPU Count	Time	Best Avg. Loss	Best Avg. Accuracy
1	5m 22s	0.2895	0.9216
2	3m 04s	0.3178	0.9286
3	2m 15s	0.3364	0.9216
4	1m 47s	0.3964	0.8718
5	1m 28s	0.3984	0.8839
6	1m 19s	0.4401	0.8397
7	1m 12s	0.4097	0.8831
8	1m 02s	0.4201	0.8875

(i) **Fastest training with >90% accuracy:** The 3-GPU configuration achieves a training time of 2m 15s with a best average accuracy of 0.9216, making it the optimal choice when both speed and accuracy constraints must be satisfied.

(ii) **Best accuracy overall:** The 2-GPU configuration yields the highest best average accuracy of 0.9286.

(iii) **Fastest runtime:** The 8-GPU configuration completes training in 1m 02s.

1.2. Qubit Count Comparison (8 GPUs, 4–8 qubits)

Using 8 GPUs, we swept the number of qubits from 4 to 8. Results are in Table 2.

Table 2: Training results across qubit counts (8 GPUs, 30 epochs, `lightning.kokkos` device).

Qubit Count	Time	Best Avg. Loss	Best Avg. Accuracy
4	1m 02s	0.4201	0.8875
5	1m 17s	0.3321	0.9062
6	1m 38s	0.4093	0.9062
7	1m 58s	0.4299	0.9125
8	2m 25s	0.3897	0.9062

More qubits do not meaningfully improve accuracy for this task, but do consistently increase runtime. Runtime scales roughly linearly with qubit count, with each additional qubit adding approximately 20–25 seconds. Accuracy remains mostly flat around 90–91%, showing no dramatic improvement from additional qubits. Loss is noisy with no clear trend. For this specific application, 4–5 qubits appears sufficient.

1.3. Shot Count Comparison (3 GPUs, 4 qubits)

We fixed the configuration to 3 GPUs and 4 qubits on the `lightning.kokkos` device for 30 epochs, varying shots from 512 to 32,768 (doubling each run) and also testing with shots disabled (`None`). Results are in Table 3.

Table 3: Convergence behavior across shot counts (3 GPUs, 4 qubits, 30 epochs). All configurations achieve a best average accuracy of 0.9216 and finish in approximately 14–15 minutes (except `None`).

Shots	Epoch of First Best Accuracy	Back-steps After First Achievement
None	9	0
512	10	4
1024	10	3
2048	10	1
4096	10	1
8192	10	0
16384	9	0
32768	9	0

(i) **General effect of shots:** All runs share the best average accuracy of 0.9216. The number of shots does not change the peak accuracy, but primarily affects the stability and speed of convergence. Other than the `None` configuration, all runs finish in 14 or 15 minutes with no clear trends in best metrics alone.

(ii) **Stability across epochs:** At low shot counts (e.g., 512), the model is most unstable—it back-steps from its best accuracy 4 times after first achieving it. As shot count increases, back-stepping decreases. At 8192 shots and above, accuracy increases monotonically. This behavior occurs because shots introduce sampling noise into the expectation value estimates of the quantum circuit; lower shot counts produce noisier gradient estimates, causing the optimizer to make less consistent updates. Higher shot counts reduce this noise and bring behavior closer to the ideal (noiseless) `None` case, which closely resembles the highest stable shot counts—consistent with noise decreasing as shots increase. For loss, across all shot counts it generally decreases continuously, with the best loss always appearing at the 29th or 30th epoch.

1.4. Classical vs. Quantum Comparison (1 GPU)

Using 1 GPU, we disabled the quantum network and compared the classical model against the quantum method. Results are in Table 4.

Table 4: Classical vs. quantum model comparison (1 GPU, 30 epochs).

Model	Time	Best Avg. Loss	Best Avg. Accuracy
Quantum (4 qubits)	5m 22s	0.2895	0.9216
Classical	1m 12s	0.2188	0.9281

The classical model is almost 5× faster to train than the quantum model and is also slightly better in both accuracy and loss. Quantum does not appear to be viable for this use case at the current scale. However, on a vastly more complex problem, the quantum model would likely have a greater

advantage. Additionally, this benchmark uses *simulated* quantum hardware—a real QPU may yield different results.

Task 2. Backend Evaluation

We loaded a pre-trained checkpoint and evaluated the validation phase using `qml_eval.py` with 1 GPU and 5 epochs across three backends. Results are shown in Table 5.

Table 5: Validation loss across 5 epochs for each backend (shots = 1024 where applicable). All backends achieve validation accuracy of 1.0000 in every epoch.

Backend	**Ep. 1**	**Ep. 2**	**Ep. 3**	**Ep. 4**	**Ep. 5**
`lightning.kokkos` (shots = None)	0.1676	0.1676	0.1676	0.1676	0.1676
`lightning.kokkos` (shots = 1024)	0.1685	0.1674	0.1681	0.1675	0.1677
IQM Quantum Computer (shots = 1024)	0.2701	0.2824	0.2643	0.2731	0.2657

All configurations achieve a validation accuracy of 1.0000 across all epochs. No training is being performed, so each validation run evaluates the same pre-trained circuit.

(i) **`lightning.kokkos`, shots disabled:** With `shots=None`, the expected value is computed analytically, yielding a fully deterministic output. Since the validation set is fixed and identical across epochs, the loss is exactly identical across all 5 epochs (0.1676).

(ii) **`lightning.kokkos`, shots = 1024:** Introducing shots adds sampling noise to the expectation value estimates, causing small epoch-to-epoch variation in loss. However, the values remain close to the theoretical `None` result, with the circuit sometimes randomly guessing slightly better or worse.

(iii) **IQM Quantum Computer, shots = 1024:** Loss on the real quantum hardware is substantially worse (around 0.27 vs. 0.17), with higher variance across epochs. Physical quantum hardware is not ideal—gate operations are imperfect and measurements are noisy. The effect is analogous to taking the pre-trained network and randomly perturbing all its weights, resulting in a model that performs measurably worse than the simulated version.

Task 3. Expanding the Dataset

We created `teamXYZ_expanded.py` by copying and modifying `qml.py` to read from the expanded dataset directory:

`/gpfs/wolf2/olcf/stf007/world-shared/9b8/hymenoptera_data_expanded`

Initially we only changed the directory path and GPU count, but we then remembered to update the number of output classes from 2 to 3 to account for the added ladybug class. The training time was slightly longer than the baseline `qml.py` run, which confirmed the changes were taking effect.

Best Average Loss: 0.5428, occurring at **epoch 30/30**.

Task 4. Quantum Takeaways

4.1. Number of Circuit Executions per Image

We called the `qml.specs` function and inspected available attributes, but were unable to extract trainable parameter counts directly from it. Through code inspection and print-statement verification, we identified the following:

For $n = 4$ qubits, the trainable parameters consist of:

- 4 parameters from `q_input_features` (the pre-net output fed into the `RY_layer`), and
- $6 \times 4 = 24$ parameters across 6 variational layers with 4 parameters each.

This gives $4 + 24 = 28$ total trainable parameters. PennyLane cannot use standard backpropagation natively on quantum circuits, so it instead uses the **parameter-shift rule**, which evaluates the circuit at two shifted angles ($+\pi/2$ and $-\pi/2$) per parameter to estimate the gradient. This results in $2 \times 28 = 56$ executions per image.

The general formula is:

$$\text{Executions} = 2 \times n_{\text{qubits}} \times (q_{\text{depth}} + 1)$$

This checks out for 5 qubits with depth 6: $2 \times 5 \times (6 + 1) = 70$.

4.2. Number of Outcome Probabilities

A single qubit can exist in a superposition of two basis states ($|0\rangle$ and $|1\rangle$). An n-qubit register can represent a superposition over 2^n basis states, each with an associated measurement probability. Therefore:

- 4 qubits $\Rightarrow 2^4 = 16$ outcome probabilities
- 5 qubits $\Rightarrow 2^5 = 32$ outcome probabilities

4.3. Viability as a QC/HPC Application

In our assessment, this is **not** a viable use case for quantum computing in the current era. Traditional classical hardware is more mature, better understood, significantly faster for this problem, and far less expensive. Our results showed the classical model was nearly $5\times$ faster to train while achieving comparable or better accuracy.

The quantum circuit is small enough to be simulated by classical hardware but as that changes the pros and cons will change. Entanglement provides interesting opportunities that classic simulation cannot replicate easily, letting it more easily learn some functions. Additionally, as the number of qubits grow, the state space increases exponentially, meaning it will be increasingly hard to simulate.